

Inkspired - Documentation

Linguet Emmanuel, Duvert Paul, Benaglia-Reullier Malo et Poirier-Meynot Balthazar

26 mars 2025

Table des matières

1	Introduction	2
1.1	Le jeu	2
1.2	Le déroulement du jeu	2
1.3	Les technologies utilisés	2
1.3.1	Les Langages de programmation	2
1.3.2	Les bibliothèques externes	3
1.3.3	Les bibliothèques standards	3
1.3.4	Les conventions de code	3
2	Structure du projet	3
2.1	Le répertoire assets	3
2.2	Le répertoire ui	4
2.3	Le répertoire scenes	4
2.4	Le répertoire utility	4
3	Organisation/Fonctionnement du jeu	4
3.1	Les rooms	4
3.1.1	Description	4
3.1.2	Représentation en BDD	4
3.1.3	RoomManager.py	5
3.2	Le dessin	6
3.2.1	Description	6
3.2.2	Représentation en BDD	6
3.2.3	GameManager.py	6
3.2.4	Canva.py	7
3.3	Les votes	8
3.3.1	Fonctionnement	8
3.3.2	Représentation en BDD	8
3.3.3	VotesManager.py	8
4	Interactions avec la base de données	9
4.1	Fonctionnement	9
4.2	Schéma Relationnel	10
4.3	Implementation	10
4.3.1	SQLProvider.py	10
4.3.2	gameInitialisation.py	11
5	Les composants	11
5.1	Button	11
5.1.1	Visuel	11
5.1.2	Button.py	11
5.2	Image	12
5.2.1	Visuel	12
5.2.2	Image.py	12
5.3	ProgressBar	12
5.3.1	Visuel	12
5.3.2	ProgressBar.py	13
5.4	Shape.py	13

5.4.1	Visuel	13
5.4.2	Shape.py	13
5.5	Text	13
5.5.1	Visuel	13
5.5.2	Text.py	13
5.6	TextInput	14
5.6.1	Visuel	14
5.6.2	TextInput.py	14
5.7	Timer	15
5.7.1	Timer.py	15
5.7.2	Visuel	15
5.8	Circle	15
5.8.1	Visuel	15
5.8.2	Circle.py	15
6	Les scènes	16
6.1	Fonctionnement	16
6.2	Implémentation	16
6.2.1	SceneManager.py	16
6.2.2	Scene.py	16
6.3	HomeScene	17
6.4	JoinRoomScene	17
6.5	InRoomScene	17
6.6	PaintingScene	18
6.7	VoteScene	18
6.8	WinnersScene	19
6.9	PodiumScene	19

1 Introduction

1.1 Le jeu

Inkspired est un jeu multijoueur en ligne basé sur le dessin, inspiré du jeu GarticPhone. Chaque joueur participe à des rounds de dessin où il doit illustrer un thème donné dans un temps imparti. L'objectif est de créer des dessins, puis de les soumettre à l'évaluation des autres joueurs.

1.2 Le déroulement du jeu

- **Création ou Rejoindre une Room** : Un joueur peut créer une nouvelle room ou rejoindre une room existante.
- **Choix des paramètres** : Le créateur de la room définit le nombre de rounds et le temps alloué à chaque dessin.
- **Phase de Dessin** : À chaque round, un thème aléatoire est assigné aux joueurs. Ils doivent dessiner en respectant le temps imparti.
- **Phase de Vote** : À la fin du temps de dessin, chaque joueur note les dessins des autres en attribuant une note de 1 à 10.
- **Détermination du Gagnant** : À la fin de tous les rounds, le joueur ayant obtenu la meilleure moyenne de votes est désigné vainqueur.

1.3 Les technologies utilisés

1.3.1 Les Langages de programmation

- **Python** : Nous avons utilisé pour ce projet le langage Python (dans la version 3.5). Ce choix est lié aux contraintes du concours, mais aussi à la possibilité d'utiliser la librairie pygame, avec laquelle quelques uns d'entre nous étions habitués.
- **MySQL/PostgreSQL** : Comme notre jeu nécessite une liaison entre les joueurs, nous utilisons le langage SQL et plus précisément MySQL. PostgreSQL est quelque peu utilisé pour la base de données permettant de jouer sur des réseaux différents (Il n'y a cependant, que très peu de différence).

1.3.2 Les bibliothèques externes

- **pygame** : «*Pygame est une bibliothèque multiplateforme libre et gratuite pour le développement d'applications multimédias telles que les jeux vidéo en utilisant Python. Elle utilise la bibliothèque Simple DirectMedia Layer et plusieurs autres bibliothèques populaires pour abstraire les fonctions les plus courantes, ce qui rend l'écriture de ces programmes plus intuitive.*» (Depuis le site de PyPI.org). Nous utilisons cette librairie pour gérer l'affichage de notre jeu.
- **mysql-connector-python** : Bibliothèque officielle de MySQL pour Python, permettant d'établir une connexion avec un serveur MySQL, d'exécuter des requêtes SQL, de gérer les transactions et d'interagir avec la base de données via une interface compatible avec DB-API. Elle prend en charge l'authentification sécurisée, la gestion des pools de connexions et la prise en charge de MySQL moderne. Nous l'utilisons pour le multijoueur de notre jeu.
- **psycopg2** : Adaptateur PostgreSQL pour Python, implémentant l'interface DB-API 2.0. Il permet d'établir des connexions avec un serveur PostgreSQL, d'exécuter des requêtes SQL, de gérer les transactions et d'interagir efficacement avec la base de données. Optimisé pour la performance, il prend en charge les fonctionnalités avancées de PostgreSQL. Nous l'utilisons pour le multijoueur online du jeu.
- **python-dotenv** : Module Python permettant de charger des variables d'environnement à partir d'un fichier .env. Il facilite la gestion des configurations en séparant les informations sensibles du code source. Compatible avec os.environ, il est couramment utilisé pour stocker des clés API, des identifiants de base de données et d'autres paramètres d'application. Nous l'utilisons pour la configuration du jeu.

1.3.3 Les bibliothèques standards

- **tempfile** : Module de la bibliothèque standard de Python permettant de créer des fichiers et des répertoires temporaires de manière sécurisée. Il offre des fonctions pour générer des fichiers temporaires anonymes, des répertoires temporaires persistants et gère automatiquement leur suppression après utilisation.
- **sys** : Module permettant d'interagir avec l'interpréteur Python et l'environnement d'exécution. Il permet de récupérer les arguments de la ligne de commande, manipuler les flux d'entrée/sortie, et gérer l'exécution du programme, comme avec sys.exit() pour quitter un programme.
- **os** : Module fournissant des fonctions pour interagir avec le système d'exploitation, comme la manipulation de fichiers et répertoires, l'accès aux variables d'environnement, et l'exécution de commandes système. Il est essentiel pour gérer les fichiers et l'environnement d'exécution.
- **time** : Module permettant de travailler avec le temps. Il offre des fonctions pour récupérer l'heure actuelle, mesurer des durées, et introduire des pauses. Il permet aussi de formater les dates et heures.
- **collections** : Module offrant des structures de données spécialisées comme *deque*, *Counter*, *OrderedDict*, et *defaultdict*. Il permet de manipuler efficacement des collections de données complexes ou d'améliorer les performances dans certaines opérations (Utilisée notamment pour l'outil du pot de peinture)

1.3.4 Les conventions de code

Pour pouvoir garder le code compréhensible et homogène, nous avons appliqué quelques conventions de code.

- **CamelCase** : La convention CamelCase est une manière d'écrire des noms de variables, de classes ou de fonctions en concaténant plusieurs mots sans espaces ni séparateurs, tout en mettant en majuscule la première lettre de chaque mot. Nous utilisons l'UpperCamelCase pour les noms des classes et des fichiers de classes.

2 Structure du projet

2.1 Le répertoire assets

Le répertoire **test** est destiné à stocker les différentes ressources externes nécessaires à l'exécution du jeu. Différents sous-répertoires sont utilisés :

- **fonts** : Ce sous répertoire contient toutes les polices d'écriture utilisées dans le jeu. Nous utilisons la police PaperNotes (<https://letterflowstudio.com>)
- **icons** : Ce sous répertoire contient toutes les images servant d'icônes.
- **images** : Ce sous répertoire contient icônes et autres éléments visuels utilisés dans le jeu (ex : objets, boutons, etc.)
- **backgrounds** : Ce sous répertoire contient les images de fond utilisées pour les différentes scènes du jeu.

Le fichier **themes.txt** est le fichier dans lequel tous les thèmes utilisés pour le jeu sont stockés.

2.2 Le répertoire ui

Ce répertoire contient tous les composants destinés à être affichés à l'écran.

2.3 Le répertoire scenes

Ce répertoire contient toutes les scènes du jeu.

2.4 Le répertoire utility

Ce répertoire contient tous les fichiers qui se chargent de toutes les actions internes au fonctionnement du jeu et qui ne sont pas affichés à l'écran.

3 Organisation/Fonctionnement du jeu

Le jeu autour de trois étapes. La première, rejoindre ce que nous avons appelé une "room", une chambre ou une partie. La seconde, le jeu en lui même, la partie dessin. Et la troisième la partie votes. Chacune de ces trois parties est représenté par ce que l'on à appelé un manager.

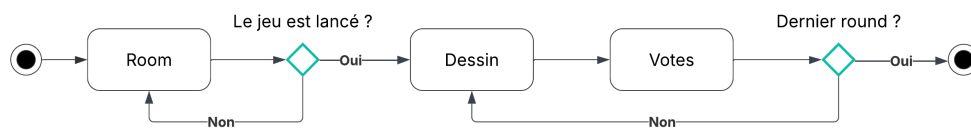


FIGURE 1 – Liaison entre les étapes

3.1 Les rooms

3.1.1 Description

Les "rooms" peuvent être considérés comme des salles de jeu. Une room correspond à une "salle" (virtuelle) où plusieurs joueurs peuvent jouer ensemble. Les joueurs doivent obligatoirement être dans une room pour pouvoir jouer. Ils peuvent soit en créer une, soit en rejoindre une. Les paramètres du jeu lui sont associés, c'est à dire que le nombre de rounds effectués et la durée de dessin sont spécifiques à la room. Un utilisateur est dit connecté lorsqu'il a rejoint une room. Une room possède un état selon qu'elle soit en attente de lancement du jeu (lobby), en jeu (playing) ou en étape de vote (voting).

3.1.2 Représentation en BDD

rooms
CP : room_id
creator
room_name
theme
state
rounds_number
round_time

TABLE 1 – Relation rooms

connected_users
PK : user_id
FK : room_id
username
rating

TABLE 2 – Relation connected_users

3.1.3 RoomManager.py

Classe RoomManager : Cette classe contient tous les outils pour créer et interagir avec les rooms.

méthode __init__(self, SQLProvider)

Initialise les attributs :

- **sqlProvider** : Instance de la classe SQLProvider de puis le paramètre.
- **username** : Nom d'utilisateur du joueur (client).
- **userId** : Identifiant de l'utilisateur quand il se connecte à une room, par défaut nul.
- **currentRoomId** : Identifiant de la room courante. Par défaut nul (quand le joueur n'a pas encore rejoint de room).
- **currentRound** : Round actuel de la partie, par défaut nul.

méthode getAllRooms(self)

Renvoie toutes les rooms existantes en BDD.

méthode getAllRoomsIds(self)

Renvoie tous les identifiants rooms existantes en BDD.

méthode getConnectedUsersNumberInRoom(self, roomId : int)

Retourne le nombres d'utilisateurs connectés à la room d'identifiant **roomId**.

méthode getCurrentRoomName(self)

Retourne le nom de la room actuelle depuis l'attribut **currentRoomId**.

méthode createConnection(self, roomId : int)

Méthode utilisée pour créer une connexion (rejoindre) avec une room.

méthode createRoom(self)

Méthode utilisée pour créer une room. Crée automatiquement une connexion à la room.

méthode closeRoom(self, roomId)

Supprime la room identifiée par **roomId**. Supprime les connexions, dessins et votes associés.

méthode closeConnection(self)

Ferme la connexion (quite) à la room courante.

méthode setRoomState(self, state : str)

Définie l'état de la room courante à **state**.

méthode setRoundsNumber(self, number : int)

Définie le nombre de rounds associés à la room courante.

méthode setRoundTime(self, time : int)

Définie le temps (en secondes) de dessin associé à la room courante.

méthode getUsersInCurrentRoom(self)

Renvoie la liste des utilisateurs connectés (qui ont rejoins) à la room courante (identifiée par l'attribut **currentRoomId**).

méthode getRoundsNumber(self)

Renvoie le nombre de rounds associés à la room courante.

méthode getRoundTime(self)

Renvoie le temps de dessin associé à la room courante.

méthode getRoomState(self)

Renvoie l'état actuel associé à la room courante.

méthode getRoomCreator(self)

Renvoie le créateur de la room courante.

méthode setUsername(self, username : str)

Définit l'attribut **username** par le paramètre **username**.

méthode doesRoomExist(self, roomId : int)

Renvoie True si la room associée à **roomId** existe et False sinon.

méthode doesUserConnectedInRoom(self, roomId : int, username : str)

Renvoie True si l'utilisateur associé au nom d'utilisateur **username** est connecté à la room associée à l'identifiant **roomId**, False sinon.

3.2 Le dessin

3.2.1 Description

La partie dessin est le point central du jeu. Le joueur doit dessiner un dessin d'un thème tiré aléatoirement pendant un temps imparti défini au début du jeu (lorsque la room est à l'état "lobby"). Les dessins sont récupérés sous formes d'images PNG puis sont ensuite encodés en hexadécimal puis stockés en BDD. Le fichier intermédiaire utilisé entre la phase de dessin et la phase d'envoi du dessin en BDD est stocké dans un répertoire temporaire (C:\TEMP pour Windows), nous utilisons pour cela la librairie tempfile (voir la section 1.3.2).

3.2.2 Représentation en BDD

drawings
FK : room_id
creator
image

TABLE 3 – Relation drawings

3.2.3 GameManager.py

Class GameManager : Cette classe contient tous les outils pour créer et interagir avec les dessins et les thèmes.

méthode __init__(self, sqlProvider, username : str, roomId : str) :

Initie les attributs :

- **sqlProvider** : Instance de SQLProvider prise depuis le paramètre **sqlProvider**
- **username** : Nom d'utilisateur du joueur (client).
- **roomId** : Identifiant de la room courante.
- **drawingTheme** : Thème de jeu, par défaut nul quand la partie n'a pas encore commencé.
- **tempdir** : Répertoire temporaire (obtenu avec tempfile).

méthode drawTheme(self)

Renvoie un thème choisis au hasard depuis la méthode **loadThemes** et l'enregistre dans la room (BDD).

méthode getTheme(self)

Récupère le thème de la room (en BDD) est le renvoie

méthode sendDrawings(self, path)

Envoie le fichier associé à **path** en BDD associé à la room courante et au joueur.

méthode deleteDrawings(self)

Supprime les dessins associés à la room courante.

méthode loadThemes(self)

Renvoie les thèmes stockés dans assets/themes.txt sous forme de liste.

méthode getBinaryArray(self, path)

Renvoie la chaîne de caractères correspondant à l'encodage hexadécimal du fichier situé à **path**.

méthode getTempDir(self)

Renvoie le répertoire temporaire enregistré dans l'attribut **tempdir**.

méthode resetTempDir(self)

Réinitialise le répertoire temporaire.

3.2.4 Canva.py

La méthode de dessin des pixels ne fonctionne pas par points pour des raisons esthétiques. Si nous avions fonctionné de la sorte, il y aurait de gros angles en cas de changement de direction du pinceau. Par conséquent nous dessinons une ligne pour une mouvement de même direction et deux cercles aux extrémités.

Class Canva : Cette classe gère une surface de dessin où l'utilisateur peut dessiner, effacer, remplir des zones de couleur, et sélectionner des couleurs. Elle permet aussi de sauvegarder et charger des dessins.

méthode __init__(self, rect : pygame.Rect, backgroundColor : pygame.Color, drawColor : pygame.Color, username : str, brushSize : int = 10) :

Initialise les attributs :

- **rect** : Rectangle définissant la taille et la position du canevas sur l'écran.
- **backgroundColor** : Couleur de fond initiale du canevas.
- **drawColor** : Couleur utilisée pour dessiner sur le canevas.
- **username** : Nom d'utilisateur du joueur (utilisé pour la sauvegarde des dessins).
- **brushSize** : Taille du pinceau utilisé pour dessiner sur le canevas (par défaut 10).
- **selectedColor** : Couleur sélectionnée par l'utilisateur pour dessiner.
- **selectedTool** : Outil sélectionné par l'utilisateur (par défaut "brush").
- **darknessValue** : Valeur de la luminosité de la couleur sélectionnée.
- **allowDraw** : Booléen permettant d'activer ou désactiver l'édition sur le canevas.

méthode update(self) :

Met à jour la surface de dessin en fonction des interactions de la souris. L'utilisateur peut dessiner, utiliser la gomme, remplir des zones avec l'outil seau, et sélectionner une couleur avec l'outil pipette.

méthode onMouseWheel(self, e) :

Gère l'événement de la molette de souris pour augmenter ou diminuer la taille du pinceau.

méthode setBrushSize(self, size : int) :

Modifie la taille du pinceau utilisé pour dessiner.

méthode setBrushColor(self, color : pygame.Color) :

Modifie la couleur utilisée pour dessiner.

méthode setSelectedTool(self, tool : "brush" | "bucket" | "colorpicker" | "eraser") :

Permet de sélectionner l'outil utilisé pour interagir avec le canevas (pinceau, seau, pipette, gomme).

méthode setSelectedColor(self, color : pygame.Color) :

Modifie la couleur sélectionnée par l'utilisateur.

méthode getSelectedColor(self) -> pygame.Color :

Retourne la couleur actuellement sélectionnée par l'utilisateur.

méthode changeDarkness(self, value : int) :

Modifie la luminosité de la couleur sélectionnée en ajustant la valeur de **darknessValue**. La luminosité est appliquée sur la couleur sélectionnée.

méthode getBrushColor(self) -> pygame.Color :

Retourne la couleur actuelle du pinceau.

méthode setBackgroundColor(self, color : pygame.Color) :

Change la couleur de fond du canevas et remplit la surface avec cette couleur.

méthode save(self, path : str) :

Sauvegarde l'image du canevas sous forme de fichier PNG dans le répertoire spécifié par **path**. Le fichier est nommé en fonction du nom d'utilisateur.

méthode load(self, path : str) :

Charge une image depuis un fichier PNG et l'affiche sur la surface du canevas.

méthode holyBucket(self, x, y, color, toile) :

Implémente un remplissage de zone (outil seau) en utilisant un algorithme de recherche en largeur. Remplit une zone de pixels ayant la même couleur de départ par la couleur spécifiée.

3.3 Les votes

3.3.1 Fonctionnement

La partie vote est nécessaire pour déterminer le ou les gagnant(s) de jeu. Chaque joueur récupère les dessins des autres joueurs et leur attribut une note de 1 à 10. Les dessins récupérés sont stockés temporairement dans un répertoire temporaire (nous utilisons tempfile, voir 1.3.2). Chaque vote(note) est stocké en BDD. A la fin du temps de vote (de notation), tous les joueurs effectuent le calcul pour déterminer le gagnant. Le gagnant est le joueur qui à la meilleur moyenne. Si deux joueur sont ex æquo, les deux joueurs ont gagnés.

3.3.2 Représentation en BDD

votes
FK : room_id
voter
attributed_vote
rating
round

TABLE 4 – Relation votes

3.3.3 VotesManager.py

Classe VoteManager : Cette classe contient tous les outils pour créer et interagir avec les dessins qui doivent êtres notés, et les votes (notes) attribués à ces dessins.

méthode __init__()

Initie les attributs :

- **sqlProvider** : Instance de SQLProvider prise depuis le paramètre **sqlProvider**
- **username** : Nom d'utilisateur du joueur (client).

- **roomId** : Identifiant de la room courante.
- **tempdir** : Répertoire temporaire (obtenu avec tempdire).
- **drawings** : La liste des chemins menant aux fichiers des dessins devant être stockés.
- **participants** : La liste des pseudonymes des joueurs

méthode `getDrawings(self)`

Récupère les dessins des joueurs stockés en BDD et les stocke dans le répertoire temporaire. Renvoie la liste des dessins.

méthode `getDrawing(self, username)`

Récupère le dessin associé à l'utilisateur **username** et le stocke dans le répertoire temporaire. Retourne les informations du dessin.

méthode `vote(self, attributedVote : str, rating : int)`

Permet de noter (voter pour) un utilisateur **username** d'une note de 1 à 10.

méthode `getVotes(self)`

Permet de récupérer la liste des votes.

méthode `getWinners(self, round : int)`

Permet de déterminer les joueurs gagnants. Renvoie la liste des joueurs qui ont gagné (qui ont obtenu la meilleure note). Si **round** est précisé, renvoie les résultats pour le round voulu.

méthode `saveDrawing(self, binary, name)`

Permet de stocker les données **binary** extraites des dessins dans le fichier **name** dans le répertoire temporaire.

4 Interactions avec la base de données

4.1 Fonctionnement

Pour la partie multijoueur de notre jeu, nous avons eu besoin de pouvoir partager les données entre les joueurs. Dans la contrainte de n'utiliser que python et SQL, nous avons essayé de nous passer de serveur pouvant faire la liaison entre les clients et avons choisi de n'utiliser que la base de données, ce qui, nous le savons n'est pas une chose facile et ouvre sur plusieurs problèmes précisés dans la section ???. Deux types de bases de données SQL sont utilisés dans ce projet. Soit une base de données MySQL pour un jeu en réseau type LAN, soit une base de données PostgreSQL pour utiliser le jeu sur tous types de réseaux. Dans la suite, nous simplifions l'écriture de "Base de données" par "BDD". Comme il n'y a pas de serveur, chaque client est en

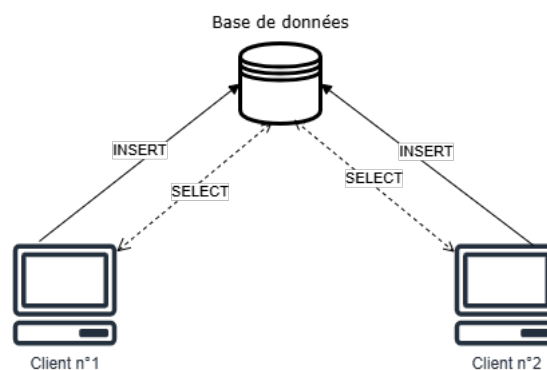


FIGURE 2 – Interactions client-BDD

liaison directe avec la BDD, chaque client effectue donc le rôle du serveur. Cette structure est organisée dans notre projet par différents fichiers.

4.2 Schéma Relationnel

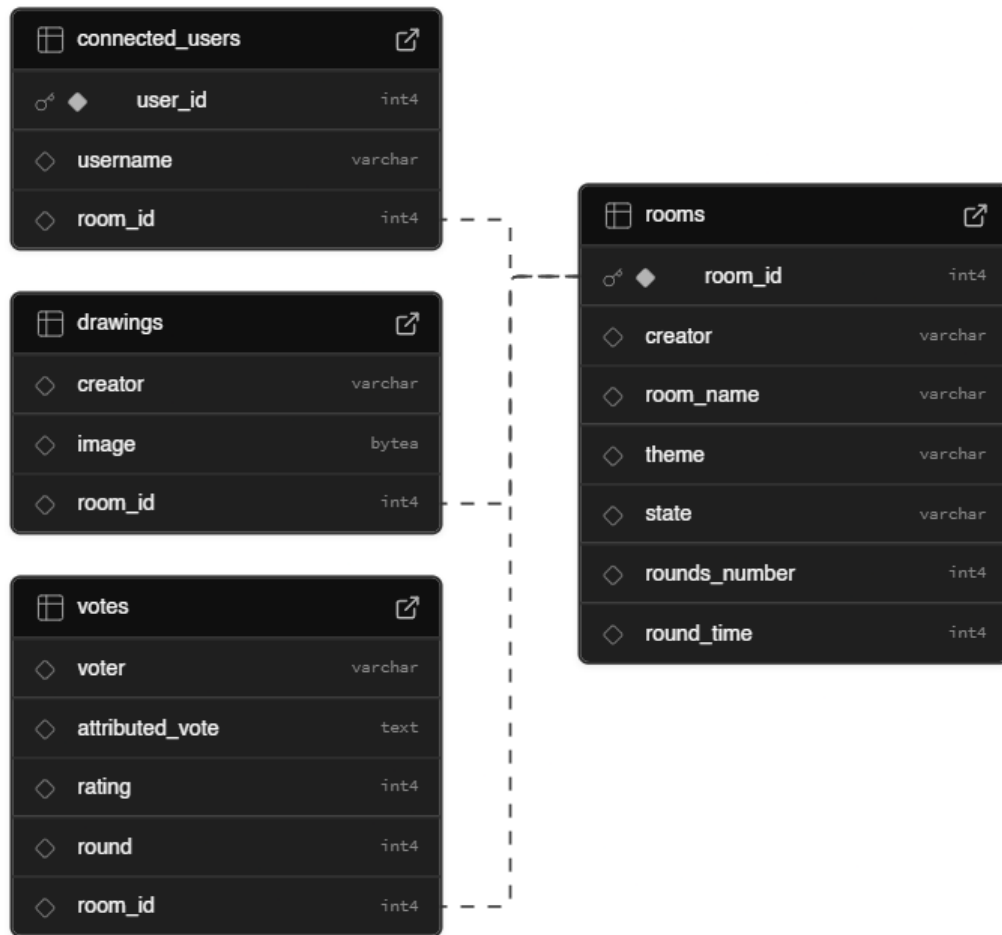


FIGURE 3 – Schéma relationnel de la BDD

`connected_users`(`user_id` : INT, `username` : VARCHAR, #`room_id` : INT)

`drawings`(`creator` : VARCHAR, `image` : bytes ou TEXT, #`room_id` : INT)

`votes`(`voter` : VARCHAR, `attributed_vote` : TEXT, `rating` : INT, #`room_id` : INT)

`room`(`room_id` : INT, `creator` : VARCHAR, `room_name` : VARCHAR, `theme` : VARCHAR, `state` : VARCHAR, `rounds_number` : INT, `round_time` : INT)

4.3 Implementation

4.3.1 SQLProvider.py

Ce fichier contient le code nécessaire pour la connexion à la base de données et son utilisation. Nous utilisons deux bibliothèques selon que l'on utilise une connexion locale ou distante. Comme expliqué plus haut nous changeons de type de BDD pour des raisons de simplicité liées à l'hébergement. Pour les BDD de données distantes nous

utilisons la librairie python `psycopg2` et pour les BDD locales nous utilisons `mysql-connector` (voir la section 1.3.2).

Classe `SQLProvider` : Classe qui initialise le `ProviderSQL`, les outils permettant de fournir la connexion sql et d'interagir avec elle.

méthode `__init__(self)`

Initie l'attribut `cnx` qui correspond à la connexion sql en fonction du type de base de données.

méthode `createDatabase(self, dbName : str)`

Crée une base de données de nom `dbName` en BDD si elle n'existe pas déjà.

méthode `useDatabase(self, dbName : str)`

Spécifie la base de donnée `dbName` comme base de donnée courante de la session.

méthode `insert(self, prompt : str, parameters : tuple returnedValue : str)`

Permet d'exécuter une commande sql `prompt` de type INSERT en spécifiant les paramètres `parameters` et retourne de la nouvelle entrée la valeur spécifiée dans `returnedValue`.

méthode `get(self, prompt : str, parameters : tuple)`

Permet d'exécuter une commande sql `prompt` de type SELECT en spécifiant les paramètres `parameters` et retourne les données demandées.

méthode `executeSQL(self, prompt : str, parameters : tuple)`

Permet d'exécuter une commande sql `prompt` en spécifiant les paramètres `parameters`.

méthode `closeConnection(self)`

Ferme la connexion avec la BDD.

4.3.2 `gameInitialisation.py`

Ce fichier contient le code permettant d'initialiser la connexion et la BDD si cela n'a pas déjà été fait par un autre client. Il est appelé pour chaque instance du jeu. Les données nécessaires à l'identification de la BDD sont prises depuis le fichier `.env` avec la librairie `dotenv` et la librairie `os` (voir les sections 1.3.2 et 1.3.3)

5 Les composants

5.1 Button

5.1.1 Visuel



FIGURE 4 – Normal



FIGURE 5 – Hovered

5.1.2 `Button.py`

Class `Button` : Cette classe représente un bouton interactif dans l'interface du jeu. Elle gère l'affichage, les interactions et les états du bouton.

méthode `__init__(self, buttonRect, action, image, hoverImage, imageCoordinates, text, fontSize=14, textColor=(0,0,0), textCoordinates=None, defaultColor=None, hoverColor=None, ErrorButton=False)` :

Initie les attributs :

- **`buttonRect` (`pygame.Rect`)** : Rectangle définissant la taille et la position du bouton.
- **`action` (`Callable`)** : Fonction appelée lorsque le bouton est cliqué.
- **`image` (`pygame.Surface` | `None`)** : Image normale du bouton (ou `None` si non utilisé).

- **hoverImage** (`pygame.Surface` | `None`) : Image du bouton lorsqu'il est survolé (ou `None` si non utilisé).
- **imageCoordinates** (`tuple[int, int]` | `None`) : Coordonnées de l'image si elle est affichée sur une surface plus grande (ou `None` si non utilisé).
- **text** (`str`) : Texte affiché sur le bouton.
- **fontSize** (`int`) : Taille de la police d'écriture du texte (par défaut 14).
- **textColor** (`pygame.Color`) : Couleur du texte (par défaut noir).
- **textCoordinates** (`tuple[int, int]` | `None`) : Coordonnées du texte sur le bouton (centré par défaut si `None`).
- **defaultColor** (`pygame.Color` | `None`) : Couleur par défaut du bouton (si aucune image n'est fournie).
- **hoverColor** (`pygame.Color` | `None`) : Couleur du bouton lorsqu'il est survolé (si aucune image de survol n'est fournie).
- **ErrorButton** (`bool`) : Indique si le bouton est un bouton d'erreur (désactive les autres boutons du même type dans la scène actuelle).

méthode disable(self)

Désactive le bouton (empêche son interaction).

méthode enable(self)

Active le bouton (permet son interaction).

méthode kill(self)

Supprime le bouton et réactive les autres boutons du même type dans la scène.

méthode update(self)

Met à jour l'état du bouton en fonction de la position de la souris et des interactions :

- Change l'image ou la couleur en fonction du survol.
- Détecte les clics et exécute l'action associée si le bouton est cliquable.
- Gère l'affichage du texte sur le bouton.

5.2 Image

5.2.1 Visuel



FIGURE 6 – Exemple d'image

5.2.2 Image.py

Class Image : Cette classe représente une image affichée dans l'interface du jeu. Elle permet de charger, redimensionner et positionner une image sur l'écran.

méthode __init__(self, image : str, imageCoordinates : pygame.Rect) :

Initie les attributs :

- **image** (`str`) : Chemin d'accès à l'image à charger.
- **imageCoordinates** (`pygame.Rect`) : Rectangle définissant la taille et la position de l'image sur l'écran.

5.3 ProgressBar

5.3.1 Visuel

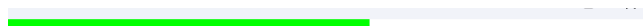


FIGURE 7 – Progress Bar

5.3.2 ProgressBar.py

Class ProgressBar : Cette classe représente une barre de progression qui peut être utilisée pour afficher une progression visuelle au cours d'un processus, par exemple pendant le temps de dessin.

méthode `__init__`(self, rect : pygame.Rect, color : pygame.Color, durationInSeconds : int, endAction : Callable) :

Initie les attributs :

- **rect (pygame.Rect) :** Rectangle définissant la taille et la position de la barre de progression.
- **color (pygame.Color) :** Couleur de la barre de progression.
- **durationInSeconds (int) :** Durée totale de la progression, en secondes.
- **endAction (Callable) :** Fonction qui sera appelée lorsque la barre de progression atteindra 100
- **value (float) :** Valeur actuelle de la progression (initialement 0).
- **startTime (float | None) :** Temps de début du processus de progression (initialement None).

méthode `update`(self)

Met à jour la barre de progression à chaque appel, en fonction du temps écoulé. Lorsque la progression atteint la durée spécifiée, la fonction **endAction** est appelée.

méthode `run_start`(self)

Démarre la barre de progression en initialisant la valeur à 0 et en enregistrant le temps de début.

5.4 Shape.py

5.4.1 Visuel

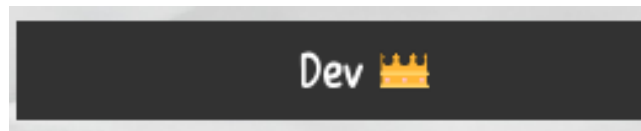


FIGURE 8 – Carte de joueur

Le module Shape.py est utilisé pour gérer l'affichage de rectangles dans l'interface utilisateur, comme par exemples les différentes cartes de joueur ou de room.

5.4.2 Shape.py

Class Shape : Cette classe représente un rectangle graphique dans l'interface du jeu. Elle permet de manipuler la taille, la couleur et l'apparence du rectangle.

méthode `__init__`(self, rect : pygame.Rect, color : pygame.Color) :

Initie les attributs :

- **rect (pygame.Rect) :** Rectangle définissant la taille et la position du rectangle.
- **color (pygame.Color) :** Couleur du rectangle.

méthode `changeColor`(self, color : pygame.Color)

Change la couleur du rectangle. La couleur est traitée pour enlever sa transparence.

méthode `changeRect`(self, rect : pygame.Rect)

Change la taille et la position du rectangle en fonction du nouveau rectangle donné. La couleur reste inchangée.

5.5 Text

5.5.1 Visuel

5.5.2 Text.py

Class Text : Cette classe représente un texte affiché dans l'interface du jeu. Elle permet de manipuler la taille, la couleur, la position et le contenu du texte.

Parties en cours : 0

FIGURE 9 – Exemple de texte

méthode `__init__(self, text : str, fontSize : int, textCoordinates : tuple[int, int], color : tuple[int, int, int], isCentered : bool = True, fontFamily : str | None = None)` :

Initie les attributs :

- **text** (**str**) : Le texte à afficher.
- **fontSize** (**int**) : Taille de la police du texte.
- **textCoordinates** (**tuple**[float, float]) : Coordonnées de position du texte. Si **isCentered** est **True**, le texte est centré à ces coordonnées, sinon il est positionné en haut à gauche.
- **color** (**pygame.Color**) : Couleur du texte en RGB.
- **isCentered** (**bool**) : Indique si le texte doit être centré autour des coordonnées données (par défaut **True**).
- **fontFamily** (**str** | **None**) : Nom de la police à utiliser. Si **None**, la police par défaut "Papernotes.ttf" est utilisée.

méthode `setText(self, text : str)`

Modifie le texte affiché. Le texte est rendu avec la même police et couleur.

5.6 TextInput

5.6.1 Visuel



FIGURE 10 – Exemple de TextInput

5.6.2 TextInput.py

Class **TextInput** : Cette classe représente un champ de saisie de texte, permettant à l'utilisateur d'entrer du texte dans une interface graphique. Elle gère l'apparence du champ de saisie, les événements de souris et de clavier.

méthode `__init__(self, rect, borderColor, selectColor, textColor, backgroundColor, placeholder = "", fontSize = 32)`

Initie les attributs :

- **rect** (**pygame.Rect**) : Rectangle définissant la taille et la position du champ de saisie.
- **borderColor** (**pygame.Color**) : Couleur de la bordure du champ de saisie.
- **selectColor** (**pygame.Color**) : Couleur de la bordure lorsque le champ de saisie est sélectionné.
- **textColor** (**pygame.Color**) : Couleur du texte saisi.
- **backgroundColor** (**pygame.Color**) : Couleur de fond du champ de saisie.
- **placeholder** (**str**) : Texte d'exemple qui apparaît lorsque le champ est vide (par défaut vide).
- **fontSize** (**int**) : Taille de la police pour le texte saisi (par défaut 32).

méthode `onMouseDown(self, e)`

Gère l'événement de clic de souris pour activer ou désactiver le champ de saisie. Si un texte d'exemple est défini, il est supprimé lorsque l'utilisateur clique dans le champ.

méthode `onKeyDown(self, e)`

Gère les événements de touche de clavier. Permet de saisir du texte, de supprimer des caractères avec la touche `backspace`, ou de réinitialiser le champ si la touche `Enter` est enfoncée.

méthode `onTextInput(self, e)`

Ajoute du texte saisi dans le champ si celui-ci est actif.

méthode `setPlaceholder(self, text)`

Change le texte d'exemple affiché dans le champ de saisie.

méthode `setText(self, text)`

Change le texte actuellement saisi dans le champ de saisie.

méthode `update(self)`

Met à jour l'apparence du champ de saisie, y compris le fond, le texte, et la bordure.

méthode `getText(self)`

Renvoie le texte actuellement saisi dans le champ.

5.7 Timer

5.7.1 Timer.py

5.7.2 Visuel



FIGURE 11 – Exemple de Timer

Class `Timer` : Cette classe représente un compte à rebours (timer) qui déclenche une action une fois la durée écoulée et met à jour un texte avec le temps restant.

méthode `__init__(self, timerDuration : int, textTimer : Text, action : Callable)`

Initie les attributs :

- **`timerDuration (int)`** : Durée du timer en secondes.
- **`textTimer (Text)`** : Instance de la classe **`Text`** utilisée pour afficher le temps restant.
- **`action (Callable)`** : Action à exécuter une fois que le timer est écoulé.

méthode `startTimer(self)`

Démarre le timer dans un nouveau thread.

méthode `timer(self)`

Méthode qui exécute le compte à rebours. Elle met à jour le texte du timer chaque seconde et appelle l'action une fois la durée écoulée.

5.8 Circle

5.8.1 Visuel

5.8.2 Circle.py

Class `Circle` : Cette classe représente un cercle dessiné sur une surface Pygame. Elle permet de changer la couleur du cercle.

méthode `__init__(self, rect : pygame.Rect, color : pygame.Color)`

Initie les attributs :

- **`rect (pygame.Rect)`** : Rectangle représentant la position et la taille du cercle.

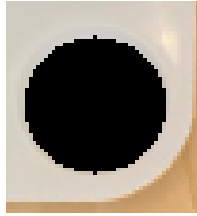


FIGURE 12 – Exemple de cercle

— **color** (`pygame.Color`) : Couleur du cercle à dessiner.

méthode `changeColor(self, color : pygame.Color)`

Change la couleur du cercle en recréant l'image du cercle avec la nouvelle couleur spécifiée.

6 Les scènes

6.1 Fonctionnement

L'affichage de notre jeu est basé sur un système de scènes. Une scène représente un état ou un environnement spécifique du jeu. C'est une unité de structuration qui définit ce qui est affiché à l'écran et comment les éléments interagissent entre eux à un moment donné. Dans notre code, une scène est en fait un groupe de sprites, un sprite étant un élément graphique. Ces deux éléments sont définis par la librairie **pygame** (voir la section 1.3.3) avec les objets `pygame.sprite.Sprite` et `pygame.sprite.Group`.

6.2 Implémentation

6.2.1 SceneManager.py

Classe `SceneManager` : Classe permettant de gérer l'affichage et l'orchestration des scènes.

méthode `__init__(self, surface : pygame.Surface)`

Initie les attributs :

- **surface** : Surface sur laquelle vont être affichés les scènes.
- **currentScene** : Instance de la scène courante. Nul par défaut.

méthode `setAsCurrentScene(self, scene : Scene, deletePreviousScene : bool)`

Définie l'instance `scene` comme scène courante. Si `deletePreviousScene = True`, la scène précédente est supprimée.

méthode `draw(self)`

Dessine la scène courante sur la surface définie dans l'attribut `surface`

méthode `update(self)`

Met à jour la scène courante.

6.2.2 Scene.py

Classe `Scene` : Classe correspondant à une scène

méthode `__init__(self)`

Initie l'attribut `spriteGroup` qui correspond au groupe de sprites (liste de sprites) de la scène.

méthode `draw(self, surface : pygame.Surface)`

Dessine la scène sur la surface `surface`.

méthode `update(self)`

Met à jour la scène.

6.3 HomeScene

La HomeScene est la scène d'accueil de notre jeu. Elle permet d'entrer son nom d'utilisateur pour jouer ainsi que d'accéder aux règles.



FIGURE 13 – Scène d'Accueil

6.4 JoinRoomScene

La JoinRoomScene est la scène permettant de rejoindre ou créer une room. On peut rejoindre une room par son id en le rentrant dans l'input ou par la liste des rooms à disposition.

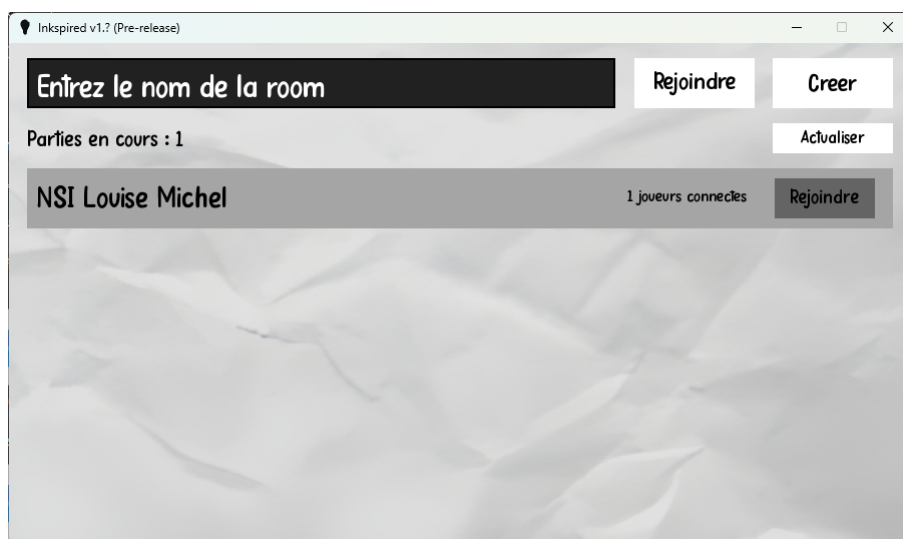


FIGURE 14 – Scène de sélection de la room

6.5 InRoomScene

La InRoomScene est la scène d'attente. Une fois la room rejoint, les joueurs attendent le début de la partie. Le créateur de la room peut modifier les paramètres de la partie comme le temps de dessin et le nombre de rounds.

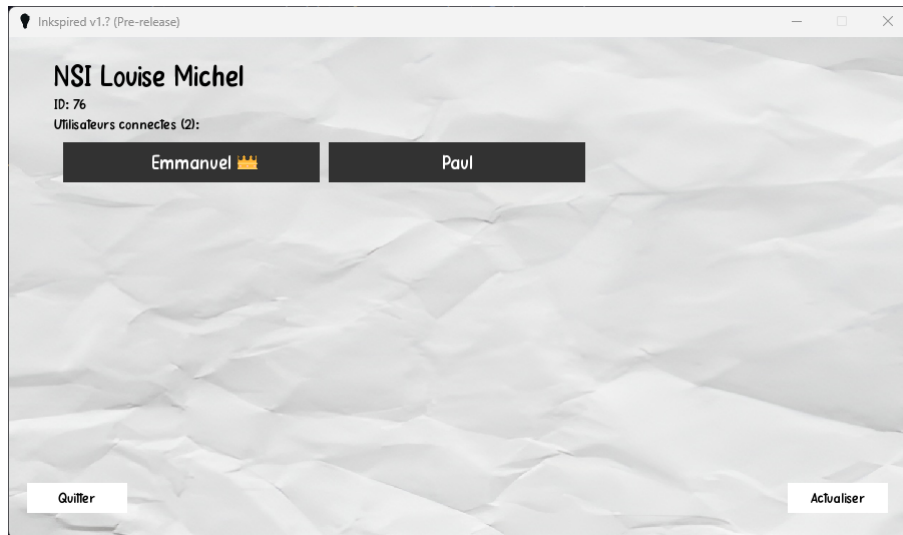


FIGURE 15 – Scène d’attente du jeu

6.6 PaintingScene

La PaintingScene est la scène de dessin. Les joueurs peuvent utiliser les outils disponibles pour dessiner un dessin selon le thème dans le temps imparti. L’outil gomme permet d’effacer une partie du dessin. L’outil Pinceau permet de dessiner. L’outil Pipette permet de récupérer une couleur depuis le dessin. L’outil seau permet de colorier une surface. La palette permet de mélanger les couleurs (avec le RGB électronique) pour obtenir une nouvelle couleur (qui s’affiche dans le sixième cercle en bas à droite).

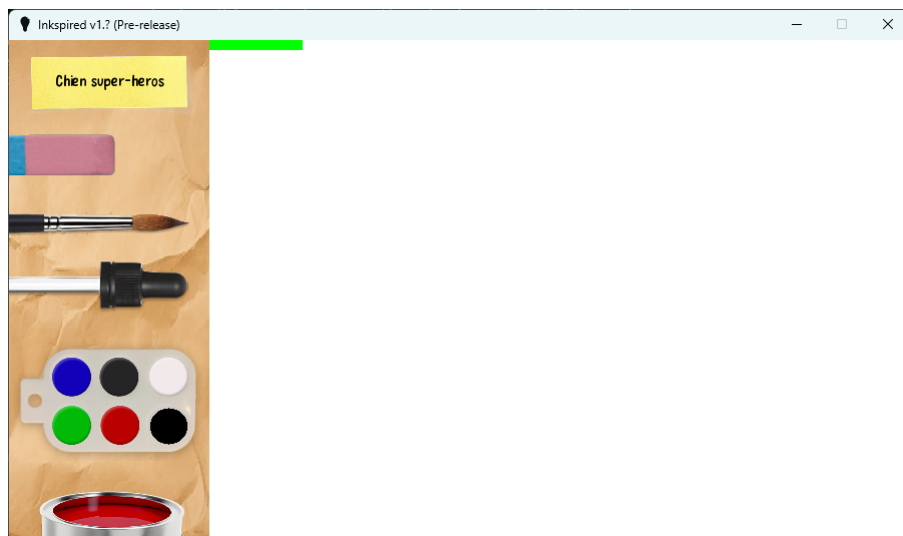


FIGURE 16 – Scène de dessin

6.7 VoteScene

La VoteScene est la scène où les joueurs peuvent noter les dessins des joueurs de 1 à 10 dans le temps imparti.

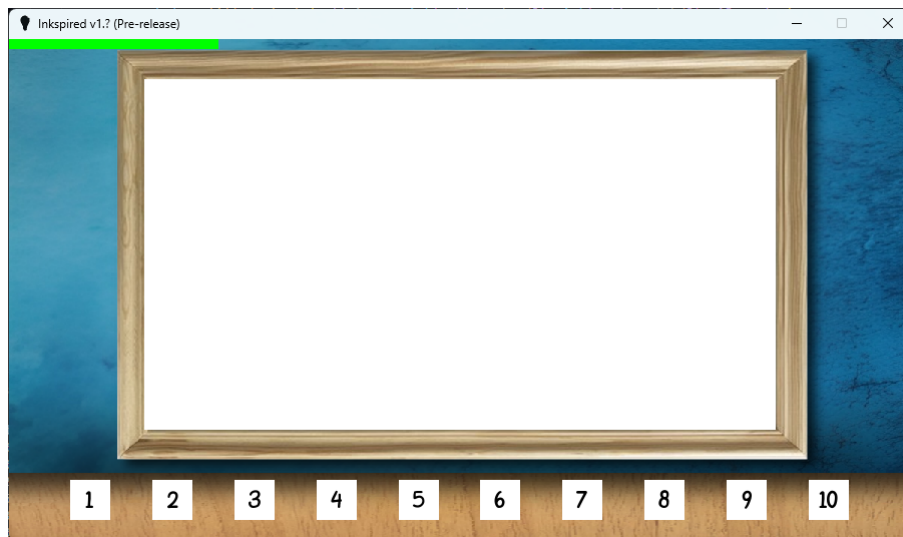


FIGURE 17 – Scène de vote

6.8 WinnersScene

La WinnerScene est la scène affichant les joueurs qui ont gagné le round actuel.

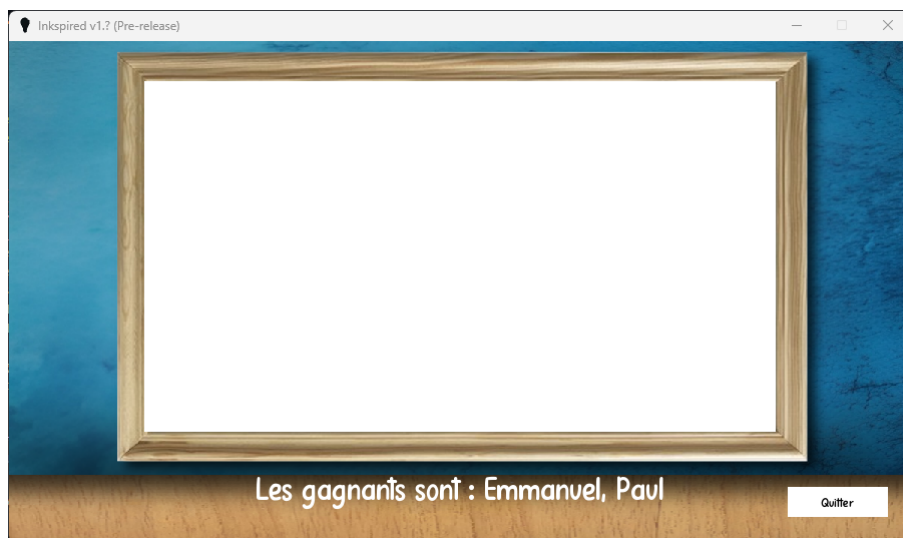


FIGURE 18 – Scène des résultats

6.9 PodiumScene

La PodiumScene est la scène qui affiche le podium classement des joueurs à la fin de la partie (quand tous les rounds sont terminés).

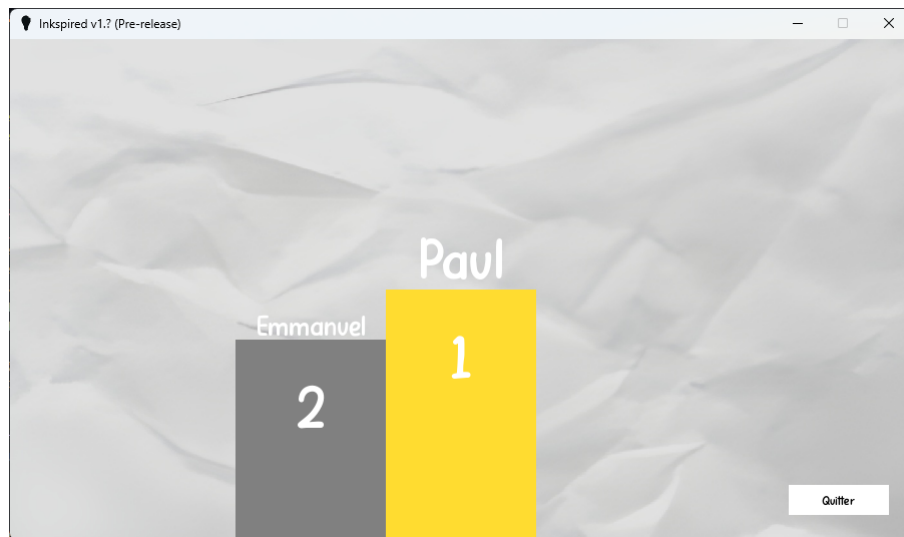


FIGURE 19 – Scène du podium